INFOMCV Assignment 3

M. Jim 1511203, L.M.M. Laan 6125573, Group number 20

March 18, 2026

1 Description and motivation of baseline model and two variants

Baseline model (CIFAR10_lenet)
The baseline model is a modified version of the classical LeNet-5 architecture adapted for color images of size $32 \times 32 \times 3$. The architecture consists of two convolutional layers followed by max-pooling, and then three fully connected layers for classification into 10 classes. Specifically, the first convolutional maps the 3 input channels to 6 feature maps of size 5×5, followed by a ReLU activation and a 2×2 max pooling layer. The second convolutional layer uses 16 filters of size 5×5, also followed by ReLU and max pooling. The resulting feature maps are flattened and passed through fully connected layers of size 120 and 84 neurons respectively, before the final output layer with 10 neurons corresponding to the CIFAR-10 classes. ReLU activations are used throughout, and Softmax is applied at the output. The weights are initialized using Kaiming uniform initialization. This model serves as a simple and well-understood baseline.

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
LeNet5                                   [1, 10]                   --
├─Conv2d: 1-1                            [1, 6, 28, 28]            456
├─MaxPool2d: 1-2                         [1, 6, 14, 14]            --
├─Conv2d: 1-3                            [1, 16, 10, 10]           2,416
├─MaxPool2d: 1-4                         [1, 16, 5, 5]             --
├─Linear: 1-5                            [1, 120]                  48,120
├─Linear: 1-6                            [1, 84]                   10,164
├─Linear: 1-7                            [1, 10]                   850
==========================================================================================
Total params: 62,006
Trainable params: 62,006
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.66
==========================================================================================
Input size (MB): 0.01
Forward/backward pass size (MB): 0.05
Params size (MB): 0.25
Estimated Total Size (MB): 0.31
==========================================================================================
```

Figure 1: Model summary of CIFAR10_lenet

Model 1 (CIFAR10_model1 - Increased filters)
This variant increases the number of convolutional filters to improve representational capacity. The first convolutional layer uses 32 filters instead of 6, and the second uses 64 instead of 16. This allows the model to capture more complex patterns and richer feature representations. The rest of the architecture remains unchanged. The motivation is that CIFAR-10 contains more complex images than datasets like MNIST, and therefore benefits from a higher-capacity feature extractor. This could lead to better classification accuracy, though it also increases the number of parameters and the risk of overfitting.

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
LeNet5_Filters                           [1, 10]                   --
├─Conv2d: 1-1                            [1, 32, 28, 28]           2,432
├─MaxPool2d: 1-2                         [1, 32, 14, 14]           --
├─Conv2d: 1-3                            [1, 64, 10, 10]           51,264
├─MaxPool2d: 1-4                         [1, 64, 5, 5]             --
├─Linear: 1-5                            [1, 120]                  192,120
├─Linear: 1-6                            [1, 84]                   10,164
├─Linear: 1-7                            [1, 10]                   850
==========================================================================================
Total params: 256,830
Trainable params: 256,830
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 7.24
==========================================================================================
Input size (MB): 0.01
Forward/backward pass size (MB): 0.25
Params size (MB): 1.03
Estimated Total Size (MB): 1.29
==========================================================================================
```

Figure 2: Model summary of CIFAR10_model1 (increased filters)

Model 2 (CIFAR10_model2 - Dropout)

This variant builds on the CIFAR10_model1 model and introduces dropout layers in the fully connected part of the network. Dropout with a probability of 0.5 is applied after each fully connected layer. The purpose is to reduce overfitting by randomly deactivating neurons during training, forcing the model to learn more robust and generalizable features. The rest of the architecture remains identical to Model 1.

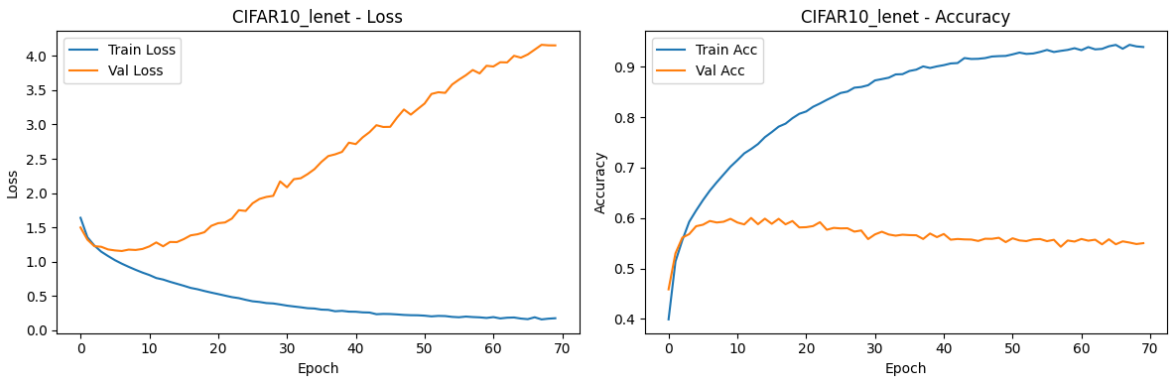
```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
LeNet5_Dropout                           [1, 10]                   --
├─Conv2d: 1-1                            [1, 32, 28, 28]           2,432
├─MaxPool2d: 1-2                         [1, 32, 14, 14]           --
├─Conv2d: 1-3                            [1, 64, 10, 10]           51,264
├─MaxPool2d: 1-4                         [1, 64, 5, 5]             --
├─Linear: 1-5                            [1, 120]                  192,120
├─Dropout: 1-6                           [1, 120]                  --
├─Linear: 1-7                            [1, 84]                   10,164
├─Dropout: 1-8                           [1, 84]                   --
├─Linear: 1-9                            [1, 10]                   850
==========================================================================================
Total params: 256,830
Trainable params: 256,830
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 7.24
==========================================================================================
Input size (MB): 0.01
Forward/backward pass size (MB): 0.25
Params size (MB): 1.03
Estimated Total Size (MB): 1.29
==========================================================================================
```

Figure 3: Model summary of CIFAR10_model2 (with dropout)

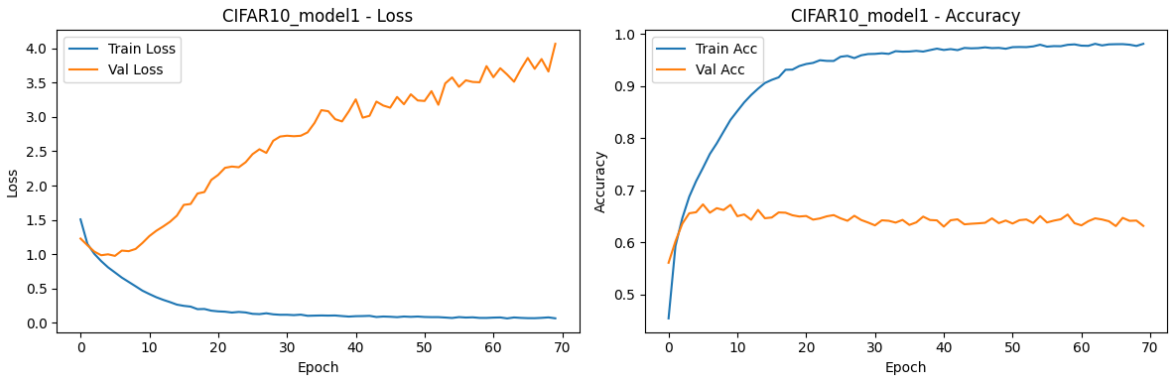
—

2 Training and validation loss for three models

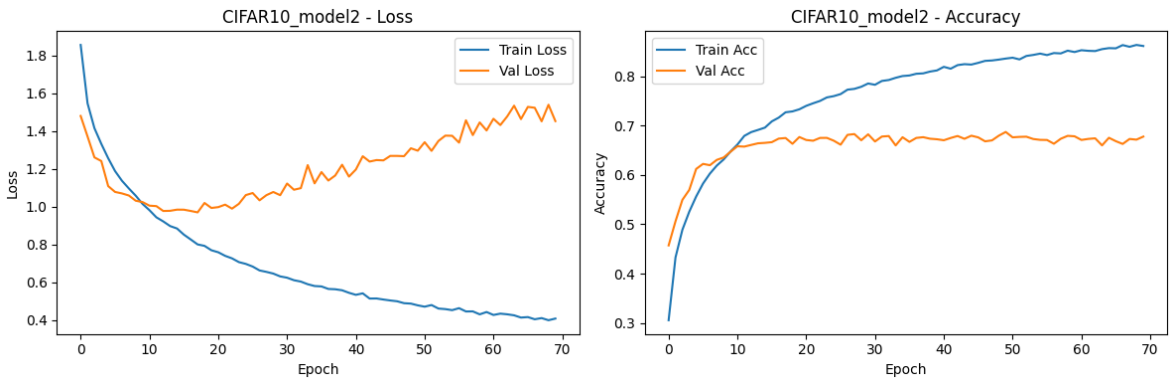
For each model, training and validation loss and accuracy were tracked over epochs. The baseline model shows steady convergence but begins to overfit after several epochs, as indicated by a gap between training and validation loss. The increased-filter model achieves lower training loss and higher validation accuracy, indicating improved learning capacity. The dropout model shows slightly higher training loss but improved generalization, as the validation loss remains closer to the training loss.



(a) LeNet Baseline



(b) LeNet model 1: Added Filters



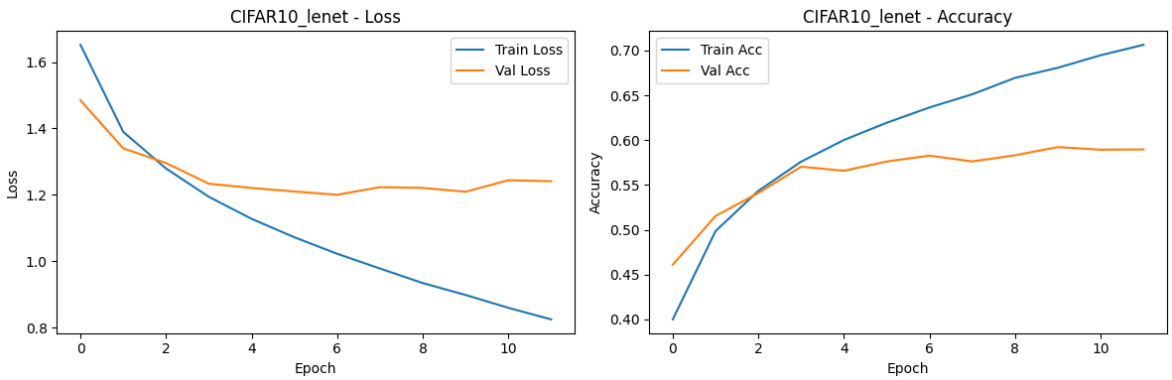
(c) LeNet model 2: With dropout

Figure 4: Training and validation performance of all models

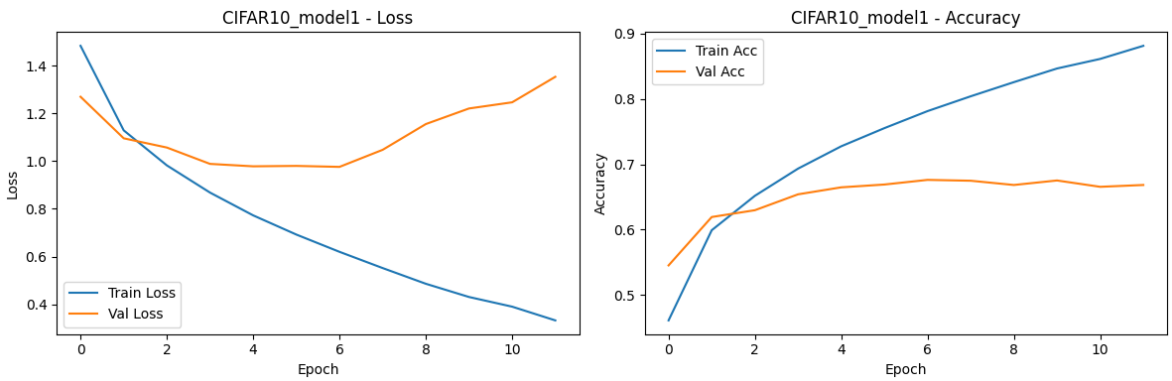
2.1 Early stop

In addition, we implemented an early stopping mechanism to prevent overfitting. The early stopping monitors the validation loss during training and stops the training process if the loss does not improve for a predefined number of epochs (patience). Specifically, we set `patience = 5`, meaning that if the validation loss does not decrease for 5 consecutive epochs, training is halted and the best model weights are restored.

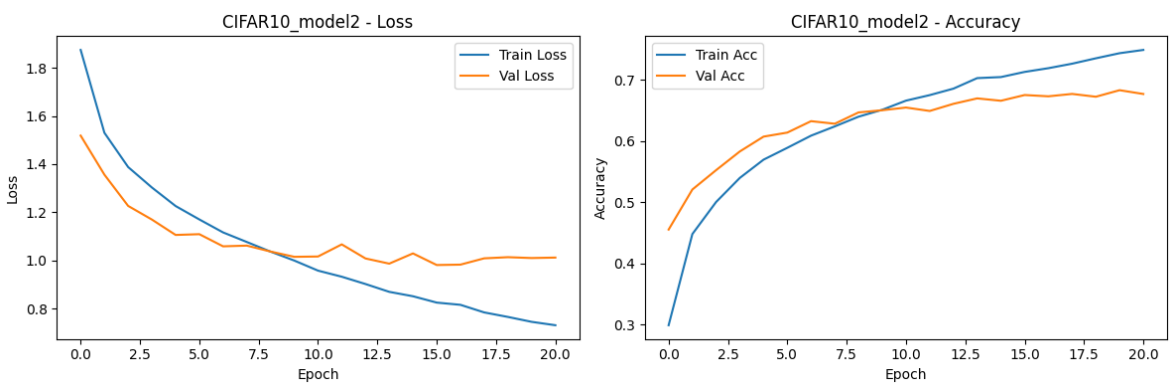
The plots in Figure 5 show that all models stopped early, as indicated by training terminating before reaching the maximum number of epochs due to the early stopping criterion. This approach helps maintain generalization by preventing overfitting, especially for the higher-capacity models.



(a) LeNet Baseline



(b) LeNet model 1: Added Filters



(c) LeNet model 2: With dropout

Figure 5: Training and validation performance of all models with early stopping (patience = 5).

3 Link to our model weights

Model weights are available at:
https://github.com/Mashu02/CV_Assignment_3_model_weights

4 Table with training and validation top-1 accuracy for all models with early stop

Model name	Training top-1 accuracy (%)	Validation top-1 accuracy (%)
CIFAR10_lenet	70.6	59.2
CIFAR10_model1	88.1	67.6
CIFAR10_model2	74.9	68.3
CIFAR10_pretrained	87.9	67.2
CIFAR100_model	78.5	65.7

Table 1: Training and validation top-1 accuracy for different models

5 Validation results in terms of model architecture

The validation results clearly reflect the impact of model architecture on performance and generalization. The baseline model, CIFAR10_lenet, has a relatively simple architecture with only two convolutional layers and a small number of filters. While it is able to learn the general structure of the CIFAR-10 images, its limited capacity leads to a noticeable gap between training and validation accuracy, indicating some underfitting and limited representational power.

CIFAR10_model1, which increases the number of convolutional filters, shows a significant improvement in training accuracy and validation performance compared to the baseline. The larger number of feature maps allows the network to capture more complex patterns and details in the images. However, the model is more prone to overfitting due to the increased number of parameters, as reflected in the slightly larger gap between training and validation metrics.

CIFAR10_model2 incorporates dropout in the fully connected layers, which effectively mitigates overfitting observed in model1. While its training accuracy is slightly higher than that of model1 due to the stochastic deactivation of neurons, the validation accuracy improves, indicating better generalization. This demonstrates that regularization techniques such as dropout are particularly beneficial for higher-capacity networks, where the risk of memorizing the training set is greater.

6 Comparison of Model Performance on the Test Set

When evaluating the models on the test set, we observe clear differences that reflect their architectural changes. The baseline model, CIFAR10_lenet, achieves moderate performance with a validation top-1 accuracy of 59.2%, which is limited by its relatively low representational capacity due to the small number of filters in the convolutional layers. Increasing the number of filters in CIFAR10_model1 significantly improves feature extraction, resulting in a higher training accuracy (88.1%) and validation accuracy (67.6%). However, this model shows a larger gap between training and validation performance, indicating that the higher-capacity network is more prone to overfitting.

In contrast, CIFAR10_model2 incorporates dropout layers in the fully connected layers to regularize the network. While this slightly reduces the training accuracy (74.9%) compared to model1, it improves generalization, as seen in a slight higher validation accuracy of 68.3%. The results demonstrate that a moderate network capacity combined with effective regularization can outperform a model on unseen data. Overall, early stopping with a patience of 5 epochs further ensures that all models avoid excessive overfitting, terminating training once the validation loss stops improving.

7 Choice tasks

7.1 Choice Task 1: Learning Rate Decay

This experiment compares the dropout CNN trained with and without a step learning-rate schedule that halves the learning rate every 5 epochs. The baseline fits the training data more strongly, reaching higher training accuracy (0.8763) and lower training loss (0.3727), but its validation loss becomes very

high (1.7443), which suggests overfitting. The scheduler model has lower training accuracy (0.8163) and higher training loss (0.5142), but better validation accuracy (0.6933) and much lower validation loss (1.0593), showing better generalization. Its best validation accuracy is also higher: 0.6955 at epoch 35 versus 0.6848 at epoch 38. Overall, the learning-rate schedule improves stability and test-time performance.

Method	**Train Acc**	**Val Acc**	**Train Loss**	**Val Loss**
Baseline	0.8763	0.6795	0.3727	1.7443
Scheduler	0.8163	0.6933	0.5142	1.0593

Table 2: Final performance comparison between baseline and learning-rate scheduler

Method	**Best Val Acc**	**Epoch**
Baseline	0.6848	38
Scheduler	0.6955	35

Table 3: Best validation accuracy comparison

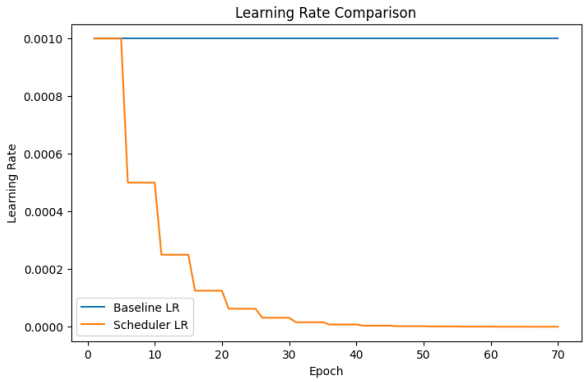


Figure 6: Learning rate decay schedule, halving every 5 epochs, compared to constant baseline.

7.2 Choice Task 2: K-fold Cross-validation

This code performs 5-fold cross-validation on the dataset using a PyTorch model. It splits the data into five parts, trains a fresh model five times, each time using four folds for training and one for validation, then saves each model and its training history. It records the best validation accuracy and loss per fold, then computes their mean and variability. The plot shows that validation accuracy increases rapidly, reaching its peak around epochs 8–12, then stabilises near 0.64–0.65, suggesting that learning has plateaued. Compared with the fixed train/validation split, cross-validation gives slightly lower accuracy but better loss, making performance estimates more reliable and robust overall.

Method	**Validation Accuracy**	**Validation Loss**
Fixed Train/Val Split	0.67920	1.017494
5-Fold Cross-Validation	0.67478	0.978613

Table 4: Comparison of validation performance between fixed split and 5-fold cross-validation

7.3 Choice Task 3: Evolutionary Hyperparameter Search

An evolutionary search strategy was employed to optimize the hyperparameters of the LeNet-5 model. The search space was defined as follows:

- **Optimizer:** Adam, SGD, RMSprop

- **Learning rate:** 10^{-3}, 5×10^{-4}, 10^{-4}
- **Weight decay:** 0, 10^{-4}
- **Batch size:** 32, 64

An initial population of six randomly sampled configurations was generated. Each configuration was trained for 15 epochs and evaluated based on its validation accuracy. The two best-performing configurations were selected as parents and mutated to produce a new generation of candidate solutions.

This evolutionary process was repeated for three generations in total.

Optimizer	Generation	Learning Rate	Weight Decay	Batch Size	Val. Accuracy
adam	3	0.0010	0.0000	64	0.7768
adam	2	0.0010	0.0000	64	0.7494
adam	3	0.0010	0.0000	64	0.7466
adam	3	0.0010	0.0001	64	0.7242
adam	2	0.0005	0.0000	64	0.7020
adam	2	0.0005	0.0000	64	0.6935
adam	3	0.0005	0.0000	64	0.6929
adam	2	0.0005	0.0000	64	0.6891
adam	3	0.0005	0.0001	64	0.6811
adam	1	0.0005	0.0000	64	0.6769
sgd	2	0.0010	0.0000	32	0.6468
sgd	1	0.0010	0.0000	32	0.6419
sgd	1	0.0010	0.0001	64	0.6161
adam	1	0.0001	0.0000	32	0.6041
sgd	2	0.0010	0.0000	64	0.6026
adam	1	0.0001	0.0000	32	0.5954
adam	1	0.0001	0.0000	64	0.5680
adam	3	0.0001	0.0000	64	0.5611

Table 5: Validation accuracy for all evaluated hyperparameter configurations using evolutionary search. Duplicate configurations may occur due to the stochastic nature of the search.

The best-performing configuration achieved a validation accuracy of **77.67%**, using the *Adam* optimizer, a learning rate of *0.001*, a weight decay of *0.0000*, and a batch size of *64*. Although RMSprop was included in the search space, it does not appear in the evaluated configurations. This is a consequence of the evolutionary search strategy, where poorly performing individuals are quickly discarded and may not be revisited. Since the population size is small and only the top-performing individuals are retained, certain configurations (such as RMSprop) may not be sufficiently explored.

7.4 Choice Task 5: Data Augmentation

To improve model generalization, several data augmentation techniques were applied to the training set with early stopping applied. The following transformations were used:

- **Random Horizontal Flip:** Applies a horizontal flip with probability 0.5.
- **Random Crop (with padding):** Crops the image to 32×32 after padding by 4 pixels, introducing translation invariance.
- **Color Jitter:** Randomly adjusts brightness, contrast, and saturation.

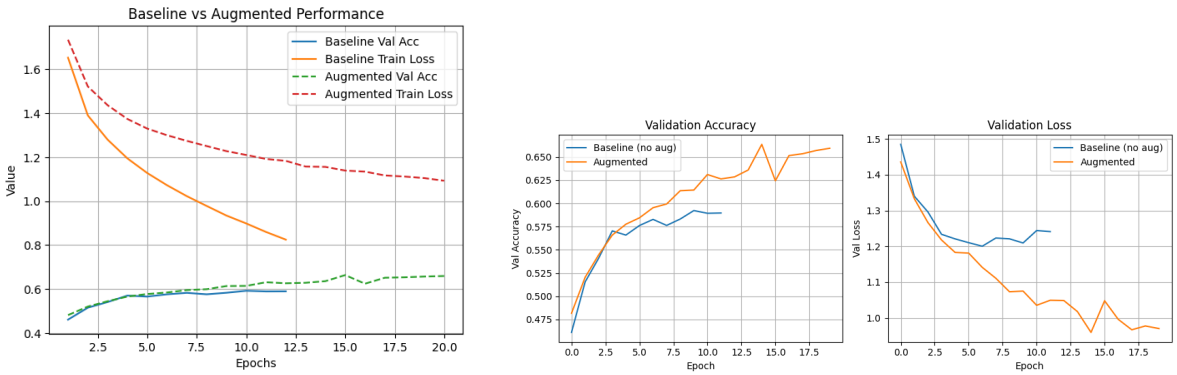
These augmentations were applied **only to the training set**, while the validation set remained unchanged.

Results

- **Baseline (no augmentation) with early stop:** 59.23%
- **With data augmentation with early stop:** 66.36%

The applied augmentations are meaningful for CIFAR-10 as they simulate real-world variations:

- **Horizontal flipping** helps the model learn that object orientation (left/right) does not change the class.
- **Random cropping** improves robustness to object position and slight translations.
- **Color jittering** helps the model generalize across different lighting conditions and color variations.



(a) Augment vs baseline

(b) Augment vs baseline

Figure 7: Comparison between baseline and augmented training

Compared to the baseline, data augmentation typically leads to better generalization by reducing overfitting. This is because the model effectively sees a larger and more diverse dataset during training.

7.5 Choice Task 6: t-SNE

The code extracts features from the penultimate fully connected layer of a trained CNN and applies t-SNE to reduce them to two dimensions for visualisation. Each point represents a test image, colored by class. The plot shows several well-formed clusters (e.g., ships, trucks, frogs), indicating the model learns meaningful representations. However, overlapping regions appear between visually similar classes, such as cats vs dogs and deer vs horses, reflecting expected confusions. Airplanes and automobiles are also partially mixed. Overall, the visualisation suggests good but imperfect class separability, aligning with the model's moderate accuracy and highlighting where misclassifications likely occur.

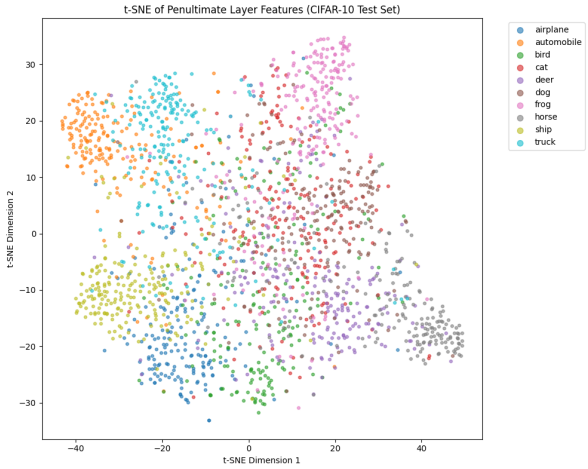


Figure 8: t-SNE of penultimate layer features on CIFAR-10 test data. Clusters show learned class structure, while overlaps indicate expected confusions between similar classes.

8 GenAI prompt

References